

ProcessPilot AI - Complete Interview Preparation Guide

Welcome to the ultimate preparation guide for **ProcessPilot AI**. This guide is structured specifically for a graduating 8th-semester engineering student. It explains the project from scratch, details the tech stack, walks through vital code segments, and provides 15 highly realistic technical interview questions and answers.

1. Project Overview: What is ProcessPilot AI?

The Core Problem

In large corporations, operational knowledge is fragmented across PDF guides, meeting transcripts, task boards, and employee directories. When an incident occurs (e.g., a server outage), employees waste valuable time searching through multiple different software applications (Slack, Google Drive, Jira) to find the correct procedure and figure out who is responsible.

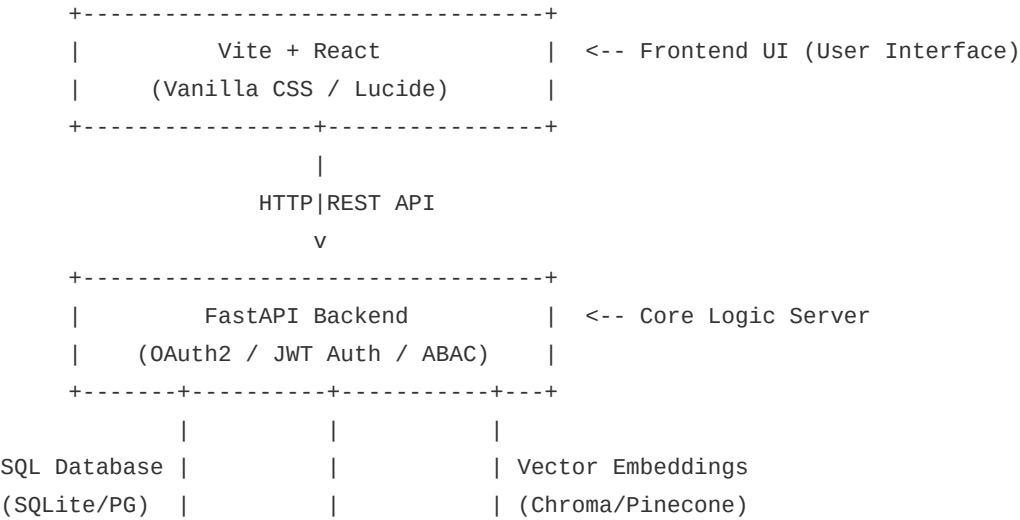
The Solution: ProcessPilot AI

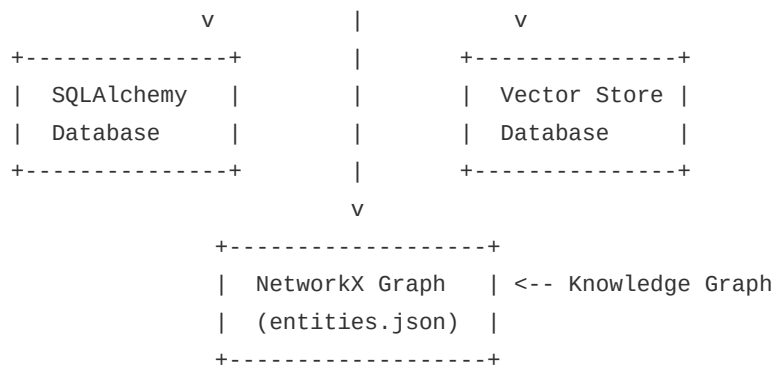
ProcessPilot AI is an **Enterprise Knowledge & Operations Copilot**. It combines:

- 1. **Multi-Agent AI**: A team of specialized AI agents (Search, Incident, SOP, Memory, CEO) that coordinate to solve complex requests.
- 2. **Graph-RAG (Retrieval-Augmented Generation)**: Uses a semantic Vector Database (Chroma/Pinecone/Qdrant) combined with a NetworkX Knowledge Graph to search not just for text, but for *relationships* between people, departments, and files.
- 3. **Operations & Hierarchy Engine**: Implements dynamic task management, meeting transcript analysis, and reporting hierarchies with strict security rules.

2. The Tech Stack: What We Used and Why

Here is the tech stack selected for ProcessPilot AI, designed to be lightweight, developer-friendly, and production-ready:





Frontend (User Interface)

- * **Vite + React**: Vite is a ultra-fast build tool for React. React allows us to build a dynamic, component-based Single Page Application (SPA).
- * **Vanilla CSS**: Used to create a dark-themed, glassmorphic design system. This demonstrates a strong handle on design tokens and clean layout styling without depending on heavy third-party CSS frameworks like Tailwind.

Backend (API Server)

- * **FastAPI**: A modern, high-performance Python web framework. It is chosen for its asynchronous execution speeds, automated documentation generation (`/docs`), and clean dependency injection system.
- * **SQLAlchemy**: A Python Object Relational Mapper (ORM) that translates Python classes into SQL database tables. It makes the backend database-agnostic.
- * **Alembic**: A database migration tool for SQLAlchemy that tracks schema modifications (like adding a column) over time without losing data.

AI & Search Databases

- * **ChromaDB / Pinecone / Qdrant**: Vector databases used to store document chunks as vector embeddings (floating-point numbers representing semantic meanings) for semantic search.
- * **NetworkX**: A Python library used to construct and analyze a Knowledge Graph in memory. It maps out corporate relationships (e.g. `*User A uploaded Document B which belongs to Department C*`).
- * **Gemini / Groq / OpenAI**: Orchestration model APIs used for parsing meeting transcripts and generating synthesis replies in the agent pipeline.

3. Vital Code Walkthroughs

Here are the most critical parts of the codebase. You must be ready to explain how these lines work.

Code Segment A: Attribute-Based Access Control (ABAC)

Traditional Role-Based Access Control (RBAC) only checks if a user is an Admin, Manager, or Employee. **ABAC** goes further by evaluating properties of the user, the resource, and the action to decide permissions.

This is implemented in ``backend/app/abac.py``:

```
def evaluate_policy(subject: User, resource_type: str, obj: Any, action: str, db: Session) -> bool:
    # 1. Admin Bypass: Admins can do anything
    if subject.role == "Admin":
        return True
```

```
# 2. Document Access Rules
if resource_type == "document":
    # Read rules
    if action == "read":
        # Managers can read their own department's documents
        if subject.role == "Manager" and obj.department_id == subject.department_id:
            return True
        # Employees can read their own department's documents
        if subject.role == "Employee" and obj.department_id == subject.department_id:
            return True
        return False

    # Delete rules
    if action == "delete":
        # Employees can delete their own uploaded documents
        if subject.role == "Employee" and obj.uploaded_by == subject.id:
            return True
        # Managers can delete documents uploaded by members of their department
        if subject.role == "Manager" and obj.department_id == subject.department_id:
            return True
        return False

# 3. Task Access Rules
if resource_type == "task":
    if action == "update":
        # Only the assignee can update a task status
        return obj.assigned_to == subject.id

return False
```

* **Why it's important**: This prevents malicious API requests where an Employee attempts to trigger a `DELETE` endpoint with a Document ID belonging to another department.

Code Segment B: Concurrent-Safe Knowledge Graph Saving (File Locking)

To persist our NetworkX knowledge graph without using an external database server, we save it to a JSON file. To prevent multi-process write errors, we built a cross-process **FileLock** in `backend/app/knowledge_graph.py`:

```
class FileLock:
    _thread_lock = threading.Lock() # Protects threads in the same process

    def __init__(self, lock_path="./knowledge_graph.json.lock", timeout=5.0):
        self.lock_path = lock_path
        self.timeout = timeout

    def __enter__(self):
        self._thread_lock.acquire()
        start_time = time.time()
        while True:
            try:
```

```
        # O_CREAT | O_EXCL is an atomic OS-level operation.
        # It fails if the file already exists, guaranteeing only one process holds the lock.
        fd = os.open(self.lock_path, os.O_CREAT | os.O_EXCL | os.O_WRONLY)
        os.close(fd)
        break
    except FileExistsError:
        if time.time() - start_time > self.timeout:
            # Break the lock if stuck/timed out
            try: os.remove(self.lock_path)
            except: pass
            time.sleep(0.05)
        return self

    def __exit__(self, exc_type, exc_val, exc_tb):
        try:
            if os.path.exists(self.lock_path):
                os.remove(self.lock_path)
        finally:
            self._thread_lock.release()
```

* **Why it's important**: When Uvicorn runs in a multi-worker production configuration, multiple processes run concurrently. Without this atomic lock, if two users uploaded files at the same time, the backend processes would write to `knowledge\_graph.json` simultaneously, corrupting the file.

Code Segment C: Process-Stable RAG Simulation Embeddings

During testing or free-tier usage, we run in "Simulation Mode". We need a way to generate deterministic vector embeddings without calling paid APIs. We solved this in `backend/app/vectorstore.py`:

```
def local_mock_embedding(text: str) -> List[float]:
    import hashlib
    # Compute a deterministic integer from the content string
    # Python's built-in hash() is process-randomized, so we use MD5 instead.
    state = int(hashlib.md5(text.encode("utf-8")).hexdigest(), 16) % 10000
    rng = np.random.default_rng(state)
    vector = rng.standard_normal(768).tolist()
    # Normalize the vector to unit length
    norm = sum(x**2 for x in vector)**0.5
    return [x/norm for x in vector] if norm > 0 else vector
```

* **Why it's important**: Python's default `hash(text)` produces different outputs every time a process restarts. By using `hashlib.md5`, we guarantee that the same query generates the exact same search vector across any process or deployment environment.

Code Segment D: Safe State Graph turn Limit

To prevent multi-agent systems from looping endlessly (costing massive API charges), we implemented a turn loop safety check in `backend/app/agents.py`:

```
class CEOAgent:
    def process_query(self, user: User, query: str, db: Session) -> Dict[str, Any]:
        # Track turns inside the session state dictionary
```

```
if user.id not in ACTIVE_AGENT_SESSIONS:
    ACTIVE_AGENT_SESSIONS[user.id] = {"turns": 0}

session = ACTIVE_AGENT_SESSIONS[user.id]
session["turns"] += 1

# Halt execution if loop count exceeds threshold
if session["turns"] > 10:
    session["turns"] = 0
    return {
        "answer": "[Warning] **Execution Halted**: Turn limit exceeded to prevent loops.",
        "sources": [],
        "steps": [{"agent": "CEOAgent", "action": "Halt", "result": "Loop detected"}]
    }
```

4. 15 Technical Interview Questions and Answers

Q1: Can you describe the overall architecture of your project?

****Answer**:**

"ProcessPilot AI is built on a split-tier architecture. The frontend is a Single Page React application built with Vite, styled with custom Vanilla CSS for a premium design system. The backend is a FastAPI server. "

>

"For data, we use a hybrid storage system: SQLAlchemy for relational transactional data (Users, Tasks, Meetings, Documents), a vector database (Chroma/Pinecone/Qdrant) to index semantic document chunks, and a NetworkX-based Knowledge Graph to query entity relationships. The backend coordinates these databases using a multi-agent system (CEOAgent, SearchAgent, GraphAgent, SOPAgent) to process user queries."

Q2: What is the difference between RAG and Graph-RAG, and how did you implement it?

****Answer**:**

"Standard RAG takes a query, runs a vector similarity search, retrieves text chunks, and feeds them to the LLM. It is great for answering specific questions about a document but misses relationships."

>

"Graph-RAG goes further by querying a knowledge graph. In my project, we look up adjacent nodes of a query entity (e.g., if a user asks about a server outage, the GraphAgent finds the server, the active SOP document, and the department members connected to it). We inject both the semantic document chunks and the graph entity connections into the LLM context. This provides the LLM with both raw information and structural organization context."

Q3: How did you implement user security and access controls?

****Answer**:**

"I implemented Attribute-Based Access Control (ABAC) in [abac.py](file:///c:/Users/KIIT/Desktop/FULL_STACK/backend/app/abac.py). FastAPI route endpoints intercept incoming

calls using dependency injection dependencies. These dependencies load the active user and target resource records from the database, evaluate them against a set of policy rules (e.g., matching the user's department ID with the document's department ID), and raise an HTTP `403 Forbidden` exception if the user does not have permission."

Q4: Why did you use SQLite for development and PostgreSQL for production, and how did you configure this?

****Answer**:**

"SQLite is a file-based database. It requires no installation, making it perfect for quick developer setup and offline testing. PostgreSQL is a robust server-based database with connection pooling and high-concurrency support."

>

"I configured this by using environment variables. In [config.py](file:///c:/Users/KIIT/Desktop/FULL_STACK/backend/app/config.py), the database URL defaults to a local SQLite file. In production, we set the `DATABASE\_URL` environment variable to a PostgreSQL connection string. SQLAlchemy handles the dialect translations automatically, making our application completely database-agnostic."

Q5: How does your application prevent file orphans during document deletions?

****Answer**:**

"In [documents.py](file:///c:/Users/KIIT/Desktop/FULL_STACK/backend/app/routes/documents.py#L126-L153), the document deletion sequence is ordered transaction-safely. We try to delete the physical file from the server disk first. If that file removal fails (due to a locked thread or permission error), we raise an HTTP 500 exception. This prevents the database transaction from calling `db.commit()`, rolling back the SQL deletions. This ensures we never have database records pointing to non-existent files, or files leaked on disk with no database record."

Q6: How does the meeting transcription and analysis feature work?

****Answer**:**

"When a user uploads a meeting transcript or inputs a link, the meeting route calls the LLM, prompting it for a structured JSON response containing a summary string and an array of action items (tasks)."

>

"We parse the JSON output using `json.loads` inside a try-except block. If the JSON is valid, we automatically create a `Meeting` database record and create the extracted `Task` records assigned to the uploader. If the LLM returns unstructured text, we fall back to a split-based text parser to prevent crash failures."

Q7: How did you solve the Chroma multi-worker database lock bottleneck?

****Answer**:**

"In Python, if a module initializes a SQLite-backed client (like ChromaDB's PersistentClient) at import time, multiple web worker processes starting up concurrently will attempt to lock the database file. This results in file locks and server crashes."

>

I solved this by implementing **Lazy Initialization**** in [vectorstore.py](file:///c:/Users/KIIT/Desktop/FULL_STACK/backend/app/vectorstore.py#L75-L81). We set the client reference to `None` at start. The actual `PersistentClient` is initialized using `self.\_init\_client()` only when a search, insert, or delete method is physically called at runtime. This defers file loading to request time, allowing the server to boot up safely.**

Q8: What are the security issues with using built-in Python `hash()` for embeddings, and how did you fix it?

****Answer**:**

*"Python's built-in `hash()` function uses hash seed randomization for security. This means every time a Python process restarts, the hash value of the same string changes. In a multi-worker server environment, Worker 1 and Worker 2 would generate different vector embeddings for the exact same text, breaking the similarity search.**

>

I resolved this by using `hashlib.md5(text.encode("utf-8")).hexdigest()` to generate a process-independent, stable seed integer, ensuring consistent embeddings across all workers and restarts.

Q9: How do you handle CORS (Cross-Origin Resource Sharing) in this project?

****Answer**:**

*"In [config.py](file:///c:/Users/KIIT/Desktop/FULL_STACK/backend/app/config.py), we define `BACKEND\_CORS\_ORIGINS` which parses a comma-separated list of allowed origins from environment variables, defaulting to local development ports. In [main.py](file:///c:/Users/KIIT/Desktop/FULL_STACK/backend/app/main.py), we pass this configuration to FastAPI's `CORSMiddleware`. This allows us to securely deploy the frontend and backend on different cloud domains without browser cross-origin blocks.**

Q10: How did you implement database migration control?

****Answer**:**

*"I integrated ****Alembic**** into the backend. I initialized the migrations environment, configured [env.py](file:///c:/Users/KIIT/Desktop/FULL_STACK/backend/alembic/env.py) to dynamically load the connection string from our app settings, and linked `target\_metadata` to our SQLAlchemy `Base.metadata`. This allows us to run `alembic revision --autogenerate` to automatically detect schema updates and `alembic upgrade head` to apply them safely in production.**

Q11: What is "Human-in-the-Loop" validation, and where is it used in your code?

****Answer**:**

*"Human-in-the-Loop validation is a design pattern where an AI agent pauses execution and requests user consent before performing critical actions. In our `CEOAgent`, if the user's query suggests creating a task, we do not execute it immediately. We save the action state to the user's active session, return a message showing the proposed details, and ask the user to respond with 'yes' or 'approve'. Only upon confirmation do we execute the database transaction.**

Q12: How did you ensure thread-safety and process-safety on your knowledge graph database?

****Answer**:**

"Since we use a local NetworkX graph saved as a JSON file, concurrent writes from multiple processes could corrupt the file. I solved this by building an atomic, cooperative `FileLock` context manager. "

>

"It utilizes Python's `threading.Lock` to block concurrent threads in the same process, and uses the OS-level `os.O\_CREAT | os.O\_EXCL` open flags to block other worker processes. Any process attempting to read or write the graph must acquire the lock file. Furthermore, we force the graph to reload from disk inside the lock block before applying modifications, ensuring data consistency."

Q13: What is the purpose of the `test\_ai\_eval.py` script?

****Answer**:**

"This script acts as our AI metric validation pipeline. It queries the CEOAgent with standard scenarios and asserts three metrics:"

- 1. *Retrieval Precision: Checks if the retrieved documents match the user's department permissions.**
- 2. *Answer Relevancy: Measures the keyword overlap between the agent's response and expected terms.**
- 3. *Faithfulness: Evaluates if the response contains hallucinated statements or API failure messages.**

"This ensures that any modifications made to our agent prompts or vector store configuration do not degrade RAG performance in production."

Q14: How are files saved on the server, and how do you prevent collisions?

****Answer**:**

"When a user uploads a document, the original name is saved in the SQL database for user-facing display. However, on the physical disk storage inside `uploads/`, we save the file using a unique UUID (`uuid.uuid4()`). This ensures that if two users upload files with matching names (e.g. `doc.pdf`), they are saved separately without overwriting each other."

Q15: If you had a budget of \$100/month, how would you scale this architecture?

****Answer**:**

"First, I would migrate the in-memory NetworkX knowledge graph to a dedicated graph database like Neo4j or use a graph table configuration inside PostgreSQL to eliminate file I/O operations."

"Second, I would use Qdrant Cloud or Pinecone to host the vector database, offloading embedding query workloads."

"Third, I would store uploaded physical documents on a cloud object store (like AWS S3 or Supabase Storage) instead of local disk storage, allowing us to run multiple stateless backend server instances behind a load balancer."